Bases de datos no estructuradas
Unidad 3: Búsqueda avanzada con MongoDB

Descargable

Índice

I. Selección de filtros y condiciones según requerimientos de búsqueda

II. Búsqueda avanzada usando expresiones regulares

III. Comandos de búsqueda en subdocumentos con iteración y asociaciones lógicas

Introducción

En esta unidad, explorarás cómo seleccionar información en MongoDB usando filtros, condiciones y documentos anidados. Aprenderás a usar patrones de expresiones regulares para búsquedas avanzadas en campos específicos.

También conocerás cómo integrar MongoDB con Python mediante la librería PyMongo, permitiéndote realizar consultas y operaciones en la base de datos de manera eficiente.

Selección de filtros y condiciones según requerimientos de búsqueda

La habilidad para organizar comandos y conformar rutinas de búsqueda en MongoDB es de suma importancia en el ámbito del manejo de datos modernos.

A medida que las organizaciones acumulan vastas cantidades de información no estructurada, la capacidad de extraer datos relevantes de manera eficiente se convierte en una competencia crítica. Este aprendizaje no solo permite optimizar el rendimiento de las consultas y mejorar la velocidad de acceso a la información, sino que también facilita la toma de decisiones informadas basadas en datos precisos y pertinentes. Además, al dominar la selección de filtros y condiciones según requerimientos específicos de búsqueda, los profesionales pueden asegurar que sus rutinas de búsqueda sean tanto precisas como eficientes, reduciendo la carga sobre los sistemas y mejorando la productividad general. En un mundo donde la información correcta en el momento adecuado puede significar una ventaja competitiva, este aprendizaje se erige como una herramienta esencial para cualquier profesional del análisis de datos y desarrollo de software.

En el manejo de bases de datos NoSQL, como MongoDB, la capacidad de organizar y estructurar comandos para conformar rutinas de búsqueda es fundamental. Esta habilidad permite a los desarrolladores y analistas de datos extraer información precisa y útil de grandes volúmenes de datos almacenados en documentos, subdocumentos y arreglos. la selección de filtros y condiciones basados en requerimientos específicos de búsqueda. La correcta implementación de estos filtros y condiciones es esencial para garantizar que las rutinas de búsqueda produzcan resultados relevantes y precisos. Los estudiantes aprenderán a identificar los requerimientos de búsqueda, seleccionar los filtros adecuados y aplicar condiciones precisas para obtener la información deseada. Este conocimiento es crucial no solo para mejorar la eficiencia en la recuperación de datos, sino también para asegurar que las consultas sean optimizadas y pertinentes a los objetivos del análisis o desarrollo en cuestión.

Filtros y condiciones: ejemplos prácticos

Se parte del siguiente contexto:

```
> use bd_Ejemplo
< switched to db bd_Ejemplo
> db.Personas.insertMany([

  { "_id" : ObjectId("5a6f05fbb56b58b447e66809"), "Nombre" : "Luís", "Apellidos" : "Martinez",
                                                  "Edad": 18, "Aficiones": [ "fútbol", "senderismo", "tenis" ],
                                                  "Amigos": [ {"Nombre" : "Monica", "Edad" : 20 }, { "Nombre" : "Andrés", "Edad" : 24 } ] },
  { "_id" : ObjectId("5a6f06fab56b58b447e6680a"), "Nombre" : "Marisa", "Apellidos" : "García",
                                                  "Edad" : 18, "Aficiones" : [ "senderismo", "tenis", "pintura" ],
                                                  "Amigos" : [ { "Nombre" : "Monica", "Edad" : 20 } ] },
  { "_id" : ObjectId("5a6f08e5b56b58b447e6680b"), "Nombre" : "Joaquin", "Apellidos" : "Rodriguez",
                                                  "Edad" : 18, "Aficiones" : [ "fútbol", "natación" ],
                                                  "Amigos" : [ { "Nombre" : "Andrés", "Edad" : 24 } ] },
  { "_id" : ObjectId("5a6f08e5b56b58b447e6680c"), "Nombre" : "Ana", "Apellidos" : "Vera",
                                                  "Edad" : 28, "Aficiones" : [ "natación", "culturismo" ],
                                                  "Amigos" : [ { "Nombre" : "Juan", "Edad" : 40 }, { "Nombre" : "Antonio", "Edad" : 52 } ] }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('5a6f05fbb56b58b447e66809'),
      '1': ObjectId('5a6f06fab56b58b447e6680a'),
      '2': ObjectId('5a6f08e5b56b58b447e6680b'),
      '3': ObjectId('5a6f08e5b56b58b447e6680c')
    }
  }
```

Consultas

1. Mostrar los amigos con edad mayor o igual a 40 años, solo se debe visualizar el apellido de la persona, el nombre y la edad del amigo.

```
> db.Personas.find({"Amigos.Edad":{$gte:40}},{_id:0, Apellidos:1,"Amigos.Nombre":1,"Amigos.Edad":1})
< {
    Apellidos: 'Vera',
    Amigos: [
      {
        Nombre: 'Juan',
        Edad: 40
      },
      {
        Nombre: 'Antonio',
        Edad: 52
      }
    ]
  }
```

2. Mostrar el primer amigo de Luis Martinez; se debe visualizar toda la información del amigo.

```
> db.Personas.find({Nombre:"Luís", Apellidos:"Martinez"},{_id:0,Nombre:0, Apellidos:0, Edad:0, Aficiones:0, Amigos:{$slice:1}})
< {
  Amigos: [
    {
      Nombre: 'Monica',
      Edad: 20
    }
  ]
}
```

3. Mostrar el segundo amigo registrado de Luis Martínez, se debe visualizar toda la información del amigo.

```
> db.Personas.find({Nombre:"Luís", Apellidos:"Martinez"},{_id:0,Nombre:0, Apellidos:0, Edad:0, Aficiones:0, Amigos:{$slice:[1,2]}})
< {
  Amigos: [
    {
      Nombre: 'Andrés',
      Edad: 24
    }
  ]
}
```

4. Mostrar nombre, apellidos, edad y nombre del amigo de las personas mayores de 25 años y cuyo amigo sea Juan.

```
> db.Personas.find({Edad:{$gt:25}, "Amigos.Nombre":"Juan"},{_id:0,Nombre:1, Apellidos:1, Edad:1})
< {
  Nombre: 'Ana',
  Apellidos: 'Vera',
  Edad: 28
}
```

5. Mostrar nombre, apellidos de las personas con afición fútbol, ordenar por apellido de forma descendente.

```
> db.Personas.find({"Aficiones":"fútbol"},{_id:0,Nombre:1, Apellidos:1}).sort({Apellidos:-1})
< {
  Nombre: 'Joaquin',
  Apellidos: 'Rodriguez'
}
{
  Nombre: 'Luís',
  Apellidos: 'Martinez'
}
```

Actualizaciones

1. Actualizar la afición de las personas que les gusta el Fútbol por Guitarra. Mostrar los cambios con find.

```
> db.Personas.updateMany({"Aficiones":"fútbol"},{$set:{"Aficiones.$":"Guitarra"}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 2,
    modifiedCount: 2,
    upsertedCount: 0
  }
```

```
> db.Personas.find({},{"Aficiones":1})
< {
    _id: ObjectId('5a6f05fbb56b58b447e66809'),
    Aficiones: [
      'Guitarra',
      'senderismo',
      'tenis'
    ]
  }
  {
    _id: ObjectId('5a6f06fab56b58b447e6680a'),
    Aficiones: [
      'senderismo',
      'tenis',
      'pintura'
    ]
  }
  {
    _id: ObjectId('5a6f08e5b56b58b447e6680b'),
    Aficiones: [
      'Guitarra',
      'natación'
    ]
  }
  {
    _id: ObjectId('5a6f08e5b56b58b447e6680c'),
    Aficiones: [
      'natación',
      'culturismo'
    ]
  }
```

2. Agregar al final de cada persona una nueva afición llamada Lectura. Mostrar los cambios con find.

```
> db.Personas.updateMany({},{$addToSet:{"Aficiones":"Lectura"}})

< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 4,
    modifiedCount: 4,
    upsertedCount: 0
  }
```

```
> db.Personas.find({},{"Aficiones":1})
< {
    _id: ObjectId('5a6f05fbb56b58b447e66809'),
    Aficiones: [
      'Guitarra',
      'senderismo',
      'tenis',
      'Lectura'
    ]
  }
  {
    _id: ObjectId('5a6f06fab56b58b447e6680a'),
    Aficiones: [
      'senderismo',
      'tenis',
      'pintura',
      'Lectura'
    ]
  }
  {
    _id: ObjectId('5a6f08e5b56b58b447e6680b'),
    Aficiones: [
      'Guitarra',
      'natación',
      'Lectura'
    ]
  }
  {
    _id: ObjectId('5a6f08e5b56b58b447e6680c'),
    Aficiones: [
      'natación',
      'culturismo',
      'Lectura'
    ]
  }
```

3. Agregar un amigo más a cada Persona al inicio de la estructura. Los nombres y edades deben ser distintos.

```
> db.Personas.updateOne({Nombre:"Luís", Apellidos:"Martinez"},{$addToSet:{"Amigos":{"Nombre":"Roberto","Edad":22}}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

```
> db.Personas.updateOne({Nombre:"Marisa", Apellidos:"García"},{$addToSet:{"Amigos":{"Nombre":"Marta","Edad":21}}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

```
> db.Personas.updateOne({Nombre:"Joaquin", Apellidos:"Rodriguez"},{$addToSet:{"Amigos":{"Nombre":"Carla","Edad":20}}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

```
> db.Personas.updateOne({Nombre:"Ana", Apellidos:"Vera"},{$addToSet:{"Amigos":{"Nombre":"Fernando","Edad":25}}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

Eliminaciones

1. Eliminar las personas que tengan edad 28 o amigos de nombre Juan.

```
> db.Personas.deleteMany({$or:[{Edad:28},{"Amigos.Nombre":"Juan"}]})
< {
    acknowledged: true,
    deletedCount: 1
  }
```

2. Eliminar las personas cuyos amigos tengan edad mayor a 23 años.

```
> db.Personas.deleteMany({"Amigos.Edad":{$gt:23}})
< {
    acknowledged: true,
    deletedCount: 2
  }
```

3. Eliminar las personas que tengan afición tenis o natación.

```
> db.Personas.deleteMany({$or:[{"Aficiones":"tenis"},{"Aficiones":"natación"}]})
< {
    acknowledged: true,
    deletedCount: 1
  }
```

Operaciones de agregaciones: ejemplos prácticos

Las operaciones de agregación son herramientas de MQL (MongoDB Query Language) que nos ayudan a procesar documentos y retornar resultados calculados.

Las operaciones de agregación se utilizan mayoritariamente para:

- Agrupar valores de varios documentos.
- Procesamiento y operaciones matemáticas.
- Analizar cambios de datos a lo largo del tiempo.

A su vez, las operaciones de agregaciones se pueden realizar de dos principales formas:

1. Tuberías y transformaciones.
2. Métodos de agregación de propósito único.

Tuberías y transformaciones

Para realizar el procesamiento de documentos, MongoDB se basa en el “Patrón de filtro de tubería”, utilizado comúnmente en arquitecturas de software y sistemas operativos Unix.

Este patrón consta de una o más etapas, en donde cada etapa realiza una operación con los datos de entrada. A su vez, la salida o resultado se la entrega a la siguiente etapa para su procesamiento.

Para aplicar este patrón, MongoDB nos entrega el método `aggregate()` . Este método recibe un array de operaciones (más conocidas como tuberías) las cuales se ejecutarán en secuencia una por una hasta llegar al resultado esperado.

Ejemplo de funcionamiento:

Estos son los operadores más utilizados en tuberías:

- Filtrado de documentos con criterios: $match.
- Orden de documentos: $sort.
- Selección de campos en específico: $project.
- Agrupación de documentos: $group.

Agrupación de documentos

Para separar los documentos en grupos según una “clave de grupo”. Se utiliza el operador $group y el resultado es un documento para cada clave de grupo única, similar a **group by** en **SQL**.

Veamos la aplicación en dos ejemplos:

Ejemplo 1

```
> use bd_agregacion
< switched to db bd_agregacion
> db.createCollection("usarios")
< { ok: 1 }
> db.usuarios.insertMany([
    {
      nombre: "Diego Gómez",
      email: "diego.g@example.com",
      edad: 27
    },
    {
      nombre: "Ana Moya",
      email: "ana.m@example.com",
      edad: 20
    },
    {
      nombre: "Juan Pardo",
      email: "juan.p@example.com",
      edad: 40
    }
  ])
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('668ff4762415081d73bb0a2e'),
      '1': ObjectId('668ff4762415081d73bb0a2f'),
      '2': ObjectId('668ff4762415081d73bb0a30')
    }
  }
```

Uso de match

1. Mostrar los usuarios que tiene 40 años.

```
> db.usuarios.aggregate([
      {$match: {edad: 40}}
  ])
< {
    _id: ObjectId('668ff4762415081d73bb0a30'),
    nombre: 'Juan Pardo',
    email: 'juan.p@example.com',
    edad: 40
  }
```

Uso de sort

2. Mostrar todos los usuarios ordenados por edad de forma descendente.

```
> db.usuarios.aggregate([
      {$sort: {edad: -1}}
  ])
< {
    _id: ObjectId('668ff4762415081d73bb0a30'),
    nombre: 'Juan Pardo',
    email: 'juan.p@example.com',
    edad: 40
  }
  {
    _id: ObjectId('668ff4762415081d73bb0a2e'),
    nombre: 'Diego Gómez',
    email: 'diego.g@example.com',
    edad: 27
  }
  {
    _id: ObjectId('668ff4762415081d73bb0a2f'),
    nombre: 'Ana Moya',
    email: 'ana.m@example.com',
    edad: 20
  }
```

Uso de proyect

3. Mostrar solo la edad y el nombre de los usuarios.

```
> db.usuarios.aggregate([
      {$project: {edad: 0, nombre: 0}}
  ])
< {
    _id: ObjectId('668ff4762415081d73bb0a2e'),
    email: 'diego.g@example.com'
  }
  {
    _id: ObjectId('668ff4762415081d73bb0a2f'),
    email: 'ana.m@example.com'
  }
  {
    _id: ObjectId('668ff4762415081d73bb0a30'),
    email: 'juan.p@example.com'
  }
```

Agrupación

4. Mostrar el promedio de las edades de los usuarios.

```
> db.usuarios.aggregate([{
        $group: {
          _id: null,
          Promedio: {$avg:  "$edad"}
        }
  }])
< {
    _id: null,
    Promedio: 29
  }
```

5. Mostrar la suma de las edades de los usuarios.

```
> db.usuarios.aggregate([{
      $group: {
        _id: null,
        Suma: {$sum:  "$edad"}
     }}])
< {
    _id: null,
    Suma: 87
  }
```

6. Mostrar la edad mínima de los usuarios.

```
> db.usuarios.aggregate([{
      $group: {
        _id: null,
        Mínimo: {$min:  "$edad"}
     }}])
< {
    _id: null,
    'Mínimo': 20
  }
```

7. Mostrar la edad máxima de los usuarios.

```
> db.usuarios.aggregate([{
      $group: {
        _id: null,
        Máximo: {$max:  "$edad"}
     }}])
< {
    _id: null,
    'Máximo': 40
  }
```

Ejemplo 2

```
> use carrera
< switched to db carrera
> db.sesiones.insertOne({nombre:"Carlos Meza", mes:"Marzo", distKm:6, tiempoMin:42})
  db.sesiones.insertOne({nombre:"Ana Moya", mes:"Marzo", distKm:10, tiempoMin:60})
  db.sesiones.insertOne({nombre:"Carlos Meza", mes:"Marzo", distKm:2, tiempoMin:12})
  db.sesiones.insertOne({nombre:"Ana Moya", mes:"Marzo", distKm:10, tiempoMin:61})
  db.sesiones.insertOne({nombre:"Carlos Meza", mes:"Abril", distKm:5, tiempoMin:33})
  db.sesiones.insertOne({nombre:"Ana Moya", mes:"Abril", distKm:42, tiempoMin:285})
  db.sesiones.insertOne({nombre:"Juan Saez", mes:"Abril", distKm:5, tiempoMin:33})

< {
    acknowledged: true,
    insertedId: ObjectId('6690218c24553161badb4092')
  }
```

1. Mostrar el número de sesiones que ha realizado cada persona.

```
> db.sesiones.aggregate(                          // aggregate significa que va a agrupar
                        [                         // lista de operaciones, a realizar en secuencia
                         {$group:                 // en este caso solo una operación, agrupar
                                 { _id:"$nombre", // agrupamos por nombre
                                   num_sesiones:  // nueva clave, num_sesiones
                                                 {$sum:1} // cuenta el número de elementos en el grupo
                                 }
                         }
                         ]
                        )

< {
    _id: 'Juan Saez',
    num_sesiones: 1
  }
  {
    _id: 'Carlos Meza',
    num_sesiones: 3
  }
  {
    _id: 'Ana Moya',
    num_sesiones: 3
  }
```

2. Mostrar el número de sesiones realizadas por mes.

```
> db.sesiones.aggregate(
  [
  {$group:
  {_id:{nombre:"$nombre",
  mes: "$mes"},
  num_sesiones: {$sum:1}
  } } ] )
< {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Marzo'
    },
    num_sesiones: 2
  }
  {
    _id: {
      nombre: 'Carlos Meza',
      mes: 'Abril'
    },
    num_sesiones: 1
  }
  {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Abril'
    },
    num_sesiones: 1
  }
  {
    _id: {
      nombre: 'Juan Saez',
      mes: 'Abril'
    },
    num_sesiones: 1
  }
```

```
{
  _id: {
    nombre: 'Carlos Meza',
    mes: 'Marzo'
  },
  num_sesiones: 2
}
```

3. Mostrar la suma de los kilómetros por persona.

```
> db.sesiones.aggregate(
  [
  {$group:
  {_id:{nombre:"$nombre"},
  num_km: {$sum:'$distKm'}
  } } ] )
< {
    _id: {
      nombre: 'Juan Saez'
    },
    num_km: 5
  }
  {
    _id: {
      nombre: 'Carlos Meza'
    },
    num_km: 13
  }
  {
    _id: {
      nombre: 'Ana Moya'
    },
    num_km: 62
  }
```

4. Mostrar los kilómetros que corre cada uno de promedio al mes.

```
> db.sesiones.aggregate(
  [
  {$group:
  { _id:{nombre:"$nombre",
  mes: "$mes"},
  media: {$avg:'$distKm'}
  } } ] )
< {
    _id: {
      nombre: 'Carlos Meza',
      mes: 'Marzo'
    },
    media: 4
  }
  {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Marzo'
    },
    media: 10
  }
  {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Abril'
    },
    media: 42
  }
  {
    _id: {
      nombre: 'Carlos Meza',
      mes: 'Abril'
    },
    media: 5
  }
```

```
{
  _id: {
    nombre: 'Juan Saez',
    mes: 'Abril'
  },
  media: 5
}
```

5. Mostrar la mayor y menor distancia recorrida por los corredores.

```
> db.sesiones.aggregate(
                        [
                         {$group:
                                 { _id:{nombre:"$nombre"},
                                   maxdist:{$max:'$distKm'},
                                   mindist:{$min:'$distKm'}
                                 }
                          }])
< {
    _id: {
      nombre: 'Juan Saez'
    },
    maxdist: 5,
    mindist: 5
  }
  {
    _id: {
      nombre: 'Carlos Meza'
    },
    maxdist: 6,
    mindist: 2
  }
  {
    _id: {
      nombre: 'Ana Moya'
    },
    maxdist: 42,
    mindist: 10
  }
```

6. Mostrar el promedio de los kilómetros mensuales de cada corredor, pero solo para aquellos valores donde el promedio sea sobre 5km.

```
> db.sesiones.aggregate( [
    {$group: { _id:{nombre:"$nombre", mes: "$mes"}, media: {$avg:'$distKm'} } },
    {$match: {media:{$gt:5}} } ] )
< {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Abril'
    },
    media: 42
  }
  {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Marzo'
    },
    media: 10
  }
```

7. Mostrar el promedio de los kilómetros por corredor y mes, ordenar por mes de forma ascendente.

```
> db.sesiones.aggregate(
  [
  {$group:
  { _id:{nombre:"$nombre", mes: "$mes"}, media: {$avg:'$distKm'}}},
  {$sort: {'_id.mes':1} }
  ] )
< {
    _id: {
      nombre: 'Juan Saez',
      mes: 'Abril'
    },
    media: 5
  }
  {
    _id: {
      nombre: 'Carlos Meza',
      mes: 'Abril'
    },
    media: 5
  }
  {
    _id: {
      nombre: 'Ana Moya',
      mes: 'Abril'
    },
    media: 42
  }
  {
    _id: {
      nombre: 'Carlos Meza',
      mes: 'Marzo'
    },
    media: 4
  }
```

```
{
  _id: {
    nombre: 'Ana Moya',
    mes: 'Marzo'
  },
  media: 10
}
```

8. Mostrar las distancias recorridas en millas, sabiendo que una milla = 1,60934 km.

```
> db.sesiones.aggregate(
                        [
                         {$project:
                                 {
                                  distMillas:{'$multiply':['$distKm',1.60934]}
                                 }
                          }])
< {
    _id: ObjectId('6690218c24553161badb408c'),
    distMillas: 9.65604
  }
  {
    _id: ObjectId('6690218c24553161badb408d'),
    distMillas: 16.0934
  }
  {
    _id: ObjectId('6690218c24553161badb408e'),
    distMillas: 3.21868
  }
  {
    _id: ObjectId('6690218c24553161badb408f'),
    distMillas: 16.0934
  }
  {
    _id: ObjectId('6690218c24553161badb4090'),
    distMillas: 8.0467
  }
  {
    _id: ObjectId('6690218c24553161badb4091'),
    distMillas: 67.59228
  }
  {
    _id: ObjectId('6690218c24553161badb4092'),
    distMillas: 8.0467
  }
```

Búsqueda avanzada usando expresiones regulares

Expresiones regulares

Las expresiones regulares (regex) son una poderosa herramienta utilizada en MongoDB para realizar búsquedas avanzadas y complejas en bases de datos NoSQL. Estas expresiones permiten a los desarrolladores y analistas de datos encontrar patrones específicos dentro de documentos, proporcionando una mayor flexibilidad y precisión en la extracción de información. Es un mecanismo muy útil para recuperar cadenas. MongoDB usa la librería Perl Compatible Regular Expression (PCRE) para gestionar las expresiones regulares de manera que cualquier expresión regular permitida por PCRE es permitida en MongoDB.

Importancia de las expresiones regulares en búsquedas. En entornos de datos grandes y no estructurados, como los que maneja MongoDB, las expresiones regulares son cruciales para satisfacer una amplia variedad de requerimientos de búsqueda. Permiten buscar y filtrar datos de manera eficiente, no solo en campos específicos, sino también a través de documentos y subdocumentos.

Aplicaciones prácticas del uso de expresiones regulares en búsquedas

1. **Búsquedas de Texto**: Encuentran palabras o frases dentro de campos de texto.
2. **Validación de Datos**: Verifican que los datos ingresados cumplan con un formato específico, como correos electrónicos o números de teléfono.
3. **Filtrado Avanzado**: Permiten buscar patrones dentro de grandes volúmenes de datos, lo que es particularmente útil en análisis de datos y minería de datos.

MongoDB incorpora soporte nativo para expresiones regulares en sus operaciones de búsqueda, lo que facilita su integración y uso directo en consultas. Esto mejora significativamente la capacidad de los desarrolladores para implementar soluciones de búsqueda robustas y eficientes en aplicaciones que dependen de bases de datos NoSQL.

Formato: { <field>: { $regex: /pattern/, $options: '<options>' } }

Patrón	Definición
/ /	Se utiliza para delimitar una expresión regular
^	Se utiliza para iniciar desde el principio el patrón
$	Se utiliza para aplicar un patrón con un carácter de término
.	Cualquier carácter
*	Cualquier carácter varias veces
[a-z]	Letra a – z
[0-9]	Números 0-9
In	Especificar más de un patrón (NO soporta añadir $regex)
<option>: i	Deshabilita sensibilidad mayúscula y minúscula

Veamos la aplicación en dos ejemplos:

Ejemplo 1

```
> use bd_expresiones1
< switched to db bd_expresiones1
> db.empleados.insertOne({nombre:'Joel López', area:'SOP-12344',edad:25, sexo:'M'})
  db.empleados.insertOne({nombre:'Patricia Silva', area:'SAS-87853',edad:35, sexo:'F'})
  db.empleados.insertOne({nombre:'Rodrigo Perez', area:'HAD-8785A',edad:33, sexo:'M'})
  db.empleados.insertOne({nombre:'Carmen Acuña', area:'R34-878',edad:39, sexo:'F'})
  db.empleados.insertOne({nombre:'Javier Maldonado', area:'SOP-12344',edad:27, sexo:'M'})
  db.empleados.insertOne({nombre:'Pamela Soto', area:'SAS-87853',edad:30, sexo:'F'})
  db.empleados.insertOne({nombre:'Ricardo Pinto', area:'HAD-8785A',edad:30, sexo:'M'})
  db.empleados.insertOne({nombre:'Carla Bustamante', area:'R34-878',edad:36, sexo:'F'})
< {
    acknowledged: true,
    insertedId: ObjectId('669027ec24553161badb409a')
  }
```

1. Mostrar los empleados cuyo nombre comience con la letra P.

```
> db.empleados.find({nombre:{$regex:/^P/}})
< {
    _id: ObjectId('669027ec24553161badb4094'),
    nombre: 'Patricia Silva',
    area: 'SAS-87853',
    edad: 35,
    sexo: 'F'
  }
  {
    _id: ObjectId('669027ec24553161badb4098'),
    nombre: 'Pamela Soto',
    area: 'SAS-87853',
    edad: 30,
    sexo: 'F'
  }
```

2. Mostrar todos los empleados cuyo nombre termine con la letra Z, sea mayúscula o minúscula; se debe visualizar solo el nombre del empleado.

```
> db.empleados.find({nombre:{$regex:/z$/i}},{_id:0,nombre:1})
< {
    nombre: 'Joel López'
  }
  {
    nombre: 'Rodrigo Perez'
  }
```

3. Mostrar los empleados donde las áreas sean SAS o SOP, visualizar solo el nombre del empleado y el área, ordenar por nombre del empleado de forma descendente

```
> db.empleados.find({area:{$in:[/^SAS/,/^SOP/]}},{_id:0,nombre:1, area:1}).sort({nombre:1})
< {
    nombre: 'Javier Maldonado',
    area: 'SOP-12344'
  }
  {
    nombre: 'Joel López',
    area: 'SOP-12344'
  }
  {
    nombre: 'Pamela Soto',
    area: 'SAS-87853'
  }
  {
    nombre: 'Patricia Silva',
    area: 'SAS-87853'
  }
```

4. Mostrar los empleados que trabajan en las áreas cuyo nombre comience con R y termine en 8; visualizar el nombre, edad y área del empleado.

```
db.empleados.find({area:{$regex:/R.*8/}},{_id:0,nombre:1, edad:1,area:1})
< {
    nombre: 'Carmen Acuña',
    area: 'R34-878',
    edad: 39
  }
  {
    nombre: 'Carla Bustamante',
    area: 'R34-878',
    edad: 36
  }
```

5. Mostrar los empleados que trabajan en las áreas cuyo nombre tenga un 5; visualizar el nombre, edad y área del empleado.

```
db.empleados.find({area:{$regex:/5/}},{_id:0,nombre:1, edad:1,area:1})
< {
    nombre: 'Patricia Silva',
    area: 'SAS-87853',
    edad: 35
  }
  {
    nombre: 'Rodrigo Perez',
    area: 'HAD-8785A',
    edad: 33
  }
  {
    nombre: 'Pamela Soto',
    area: 'SAS-87853',
    edad: 30
  }
  {
    nombre: 'Ricardo Pinto',
    area: 'HAD-8785A',
    edad: 30
  }
```

6. Mostrar los empleados de las áreas SAS y SOP; visualizar el nombre, edad y área del empleado.

```
db.empleados.find({area:{$regex:/[S][A|O][S|P]*/}},{_id:0,nombre:1, area:1})
< {
    nombre: 'Joel López',
    area: 'SOP-12344'
  }
  {
    nombre: 'Patricia Silva',
    area: 'SAS-87853'
  }
  {
    nombre: 'Javier Maldonado',
    area: 'SOP-12344'
  }
  {
    nombre: 'Pamela Soto',
    area: 'SAS-87853'
  }
```

7. Mostrar los empleados cuyo departamento comience con tres letras; visualizar el nombre y área del empleado.

```
db.empleados.find({area:{$regex:/^[A-Z][A-Z][A-Z]*/}},{_id:0,nombre:1, area:1})
< {
    nombre: 'Joel López',
    area: 'SOP-12344'
  }
  {
    nombre: 'Patricia Silva',
    area: 'SAS-87853'
  }
  {
    nombre: 'Rodrigo Perez',
    area: 'HAD-8785A'
  }
  {
    nombre: 'Javier Maldonado',
    area: 'SOP-12344'
  }
  {
    nombre: 'Pamela Soto',
    area: 'SAS-87853'
  }
  {
    nombre: 'Ricardo Pinto',
    area: 'HAD-8785A'
  }
```

8. Mostrar los empleados cuyo departamento comience con una letra y el segundo carácter sea número; visualizar el nombre y área del empleado.

```
db.empleados.find({area:{$regex:/^[A-Z][0-9]/}},{_id:0,nombre:1, area:1})
< {
    nombre: 'Carmen Acuña',
    area: 'R34-878'
  }
  {
    nombre: 'Carla Bustamante',
    area: 'R34-878'
  }
```

9. Mostrar los empleados cuyo nombre de área comience con 3 letras y termine con letra o número, además debe ser mayor a 30 años; visualizar el nombre, edad y área del empleado.

```
> db.empleados.find({$and:[{area:{$regex:/^[A-Z][A-Z][A-Z].*[A|0-9]/}}, {edad:{$gt:30}}]},{_id:0,nombre:1, area:1, edad:1})
< {
    nombre: 'Patricia Silva',
    area: 'SAS-87853',
    edad: 35
  }
  {
    nombre: 'Rodrigo Perez',
    area: 'HAD-8785A',
    edad: 33
  }
```

Comandos de búsqueda en subdocumentos con iteración y asociaciones lógicas

Trabajando con Python y MongoDB

PyMongo es el paquete de controladores oficial para controlar MongoDB con Python, es decir, permite unir estas dos tecnologías. Desde la consola de MongoDB, se tiene control absoluto de sobre las bases de datos, colecciones y documentos; sin embargo, cuando se quiere usar

esta base de datos con aplicaciones o script desarrollados en Python, se necesita un intermediario para realizar distintas operaciones.

Un intermediario que puede proporcionar esta comunicación es PYMONGO, una librería que sirve como para trabajar con Python y MongoDB. Revisemos su instalación y las primeras acciones allí.

Instalación de pymongo

- Se debe ejecutar pip install pymongo desde el terminal.}
- Luego se debe ingresar a MongoDB, por ejemplo, con Compass Haciendo clic en Connect
- Se abre cualquier editor de Python
- Se realizan las primeras acciones que son de importación y conexión:
- Luego se crea una nueva base de datos y se muestra su creación, en este caso no mostrará la base de datos creada porque esta está vacía
- Luego se crea la colección
- Se puede verificar que la base de datos ha sido creada con:

```
#Muestra todas las colecciones base de datos bd_ejemplo
print(db.list_collection_names())
```

- Se crea ahora un documento en la colección:

```
coleccion.insert_one({
        'nombre':'Leonardo Zuñiga',
         'edad':20,
         'intereses':['Musica','Guitarra']

   })
```

- Se verifica en MongoDB, por ejemplo, en la Shell:

```
> show dbs
< admin        40.00 KiB
  bd_ejemplo   40.00 KiB
  config      108.00 KiB
  local       112.00 KiB
```

- Se verifica en la base de datos bd_ejemplo

```
> use bd_ejemplo
< switched to db bd_ejemplo
> show collections
< personas
```

```
> db.personas.find()
< {
    _id: ObjectId("668ec7a3a133aba8cc24c83e"),
    nombre: 'Leonardo Zuñiga',
    edad: 20,
    intereses: [
      'Musica',
      'Guitarra'
    ]
  }
```

- En Python para ver la cantidad de documentos de la colección:

```
print(coleccion.count_documents({}))
```

- Para insertar más de un documento desde Python:

```
coleccion.insert_many([
{
        'nombre':'Marta Opazo',
        'edad':25,
        'intereses':['Canto','Baile']
},
{
        'nombre':'Victor Paredes',
        'edad':22,
        'intereses':['Deporte','Cine']
},
{
        'nombre':'Marta Opazo',
        'edad':25,
        'intereses':['Canto','Baile']
},
{
        'nombre':'Luis Cardenas',
        'edad':28,
        'intereses':['Deporte','Cine']
}
])
```

Consultas desde Python

Para mostrar todos los documentos desde Python:

```
for documentos in coleccion.find({}):
    print (documentos)
```

Para mostrar todas las personas con edad mayor a 25 años:

```
for documentos in coleccion.find({"edad":{"$gt":25}}):
    print (documentos)
```

Para mostrar el primer documento donde las personas tengan edad menor a 25 años:

Ahora la consulta no entrega un arreglo por lo que no es necesario recorrer, se guarda en una variable llamada, por ejemplo, doc.

```
doc = coleccion.find_one({"edad":{"$lt":25}})
print (doc)
```

Eliminaciones

Para eliminar la primera persona registrada que tiene 20 años:

```
coleccion.delete_one({"edad":20})
```

Para verificar:

```
doc = coleccion.find_one({"edad":20})
print (doc)
```

Actualizaciones

Actualizar la edad de las personas de 24 a 25 años:

```
coleccion.update_many({"edad":24},{"$set":{"edad":25}})
```

Para verificar:

```
for documentos in coleccion.find({}):
    print (documentos)
```

Con subdocumentos

1. **Importación de librerías:** Para interactuar con MongoDB desde Python, utilizamos la librería pymongo. Primero, debemos instalar la librería (si no está instalada) y luego importarla en nuestro script de Python.
2. **Creación de la conexión:** Después de importar pymongo, el siguiente paso es crear una conexión a nuestra base de datos MongoDB. Para esto, necesitamos la URL de conexión de nuestra instancia de MongoDB.
3. **Selección de la Base de Datos:** Una vez establecida la conexión, seleccionamos la base de datos que vamos a utilizar. Si la base de datos no existe, MongoDB la creará automáticamente cuando insertamos el primer documento.
4. **Selección de la colección:** Dentro de la base de datos, seleccionamos la colección donde almacenaremos nuestros documentos. Similar a la base de datos, si la colección no existe, MongoDB la creará cuando insertamos el primer documento.

```
from pymongo import MongoClient

cliente =  MongoClient('localhost')

db = cliente['bd_empresa']

coleccion= db['Trabajadores']
```

Inserción de datos desde Python

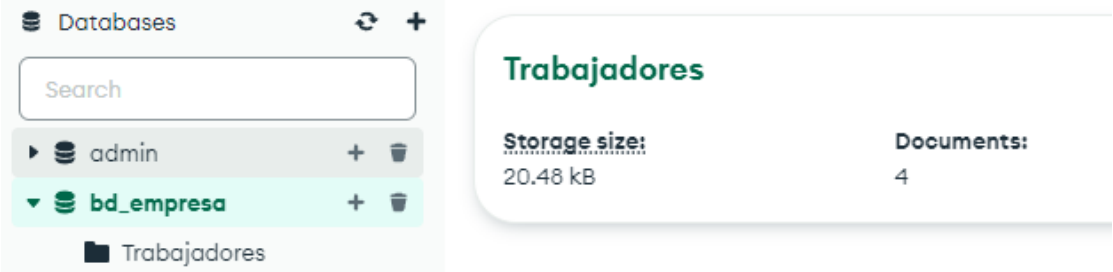
```
coleccion.insert_one({
    "nombre" : "Roberto",
    "apellido" : "Canales",
    "edad" : 35,
    "sede" : "Santiago",
    "puesto" : "Desarrollo",
    "gastos" : [
        {
            "fecha" : "Enero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 100000
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 180000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 20000
                }
            ]
        },
        {
            "fecha" : "Febrero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 0
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 105000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 15000
                }
            ]
        }
    ]
})
```

```
coleccion.insert_one({
    "nombre" : "Maria",
    "apellido" : "Torres",
    "edad" : 43,
    "sede" : "Rancagua",
    "puesto" : "Operaciones",
    "gastos" : [
        {
            "fecha" : "Enero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 150000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 20000
                }
            ]
        },
        {
            "fecha" : "Febrero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 4000
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 120000
                }
            ]
        }
    ]
}
)
```

```
coleccion.insert_one({
    "nombre" : "Luis",
    "apellido" : "Caceres",
    "edad" : 28,
    "sede" : "Rancagua",
    "puesto" : "Operaciones",
    "gastos" : [
        {
            "fecha" : "Enero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 150000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 8000
                }
            ]
        },
        {
            "fecha" : "Febrero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 20000
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 140000
                }
            ]
        }
    ]
}
)
```

```
coleccion.insert_one({
    "nombre" : "Fernanda",
    "apellido" : "Poblete",
    "edad" : 30,
    "sede" : "Santiago",
    "puesto" : "Desarrollo",
    "gastos" : [
        {
            "fecha" : "Enero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 0
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 200000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 5000
                }
            ]
        },
        {
            "fecha" : "Febrero",
            "Comercio" : "Supermercado",
            "Compra" : [
                {
                    "Concepto" : "Electronica",
                    "Costo" : 15000
                },
                {
                    "Concepto" : "Alimentación",
                    "Costo" : 180000
                },
                {
                    "Concepto" : "Deportes",
                    "Costo" : 10000
                }
            ]
        }
    ]
}
)
```

Se ejecuta y verifica en MongoDB Compass:



Consultas

1. Mostrar todos los trabajadores con gastos 0 en cualquier tipo de compra; visualizar solo el nombre, apellido puesto y edad.

```
for documentos in coleccion.find({"gastos.Compra.Costo":0
                                  },
                                 {"_id":0,"nombre":1, "apellido":1, "puesto":1, "edad":1}):
   print (documentos)
```

2. Mostrar todos los trabajadores con edad 30 años y con gastos en cualquier tipo de compra mayores o iguales a 180000; visualizar solo el nombre, apellido puesto y edad.

```
for documentos in coleccion.find({"gastos.Compra.Costo":{"$gte":180000}, "edad":30
                                  },
                                 {"_id":0,"nombre":1, "apellido":1, "puesto":1, "edad":1}):
 print (documentos)
```

3. Mostrar todos los trabajadores con gastos de 4000 en cualquier tipo de compra y sea de la sede Rancagua; visualizar solo el nombre, apellido puesto y sede.

```
for documentos in coleccion.find({"gastos.Compra.Costo":4000, "sede":"Rancagua"
                                  },
                                 {"_id":0,"nombre":1, "apellido":1, "puesto":1, "sede":1}):
 print (documentos)
```

En síntesis, has aprendido que aplicar filtros y condiciones en consultas de MongoDB es esencial para el análisis de datos y desarrollo de software. MongoDB permite crear búsquedas y documentos embebidos, y realizar búsquedas avanzadas con expresiones regulares.

También aprendiste a usar MongoDB con Python mediante PyMongo. MongoDB puede conectarse con varios lenguajes de programación, y es crucial tener una conexión adecuada para realizar operaciones de manera óptima y confiable, asegurando la eficiencia en consultas, eliminaciones y actualizaciones.

Bibliografia

- **Iglesias, A; Cuadra, D; Castro, E.** (2014). Desarrollo de bases de datos. RA-MA Editorial.
- **Sarasa, A.** (2026). *Introducción a las bases de datos NoSQL usando MongoDB*. Editorial UOC. https://elibro.net/es/lc/inacap/titulos/58524